Organização de Computadores e Sistemas Operacionais

Projeto Semestral Avaliação 3

ARTIGO, TRABALHO PRÁTICO E APRESENTAÇÃO

Objetivos do projeto:

Este trabalho visa estimular o pensamento crítico e a solução de problemas na área de sua formação utilizando conceitos de Organização de Computadores e Sistemas Operacionais adquiridos na UC durante o semestre.

Objetivo específicos:

- - Apresentar um estudo que contemple os conceitos observados na UC de Organização de Computadores e Sistemas Operacionais na forma da apresentação e implementação de um estudo de caso e artigo;

  - Elaborar um artigo contendo a apresentação das tecnologias, do tema escolhido, área de aplicação, o embasamento teórico, o projeto, arquitetura de sistemas operacionais, as tecnologias usadas, por fim, a discussão final com possíveis resultados obtidos e conclusões e sugestões de trabalhos futuros.

Organização de Computadores e Sistemas Operacionais

Regras:

1) Os trabalhos serão em grupos de 5 alunos. Somente será permitida a participação de alunos regularmente matriculados. A troca de grupo será permitida somente após **autorização do professor**. Só receberão notas os alunos cujos nomes estiverem no artigo enviado pela equipe
Obs: Não será permitido grupo com apenas um aluno.

2) A pontuação total do projeto será de 30 pontos;
Apresentação presencial na sala: (0 à 10 pontos).
Documentação constando: do roteiro da implementação; e Artigo; (0 à 10 pontos).
e implementação ou simulação do protótipo (0 à 10 pontos).

3) Os temas dos trabalhos serão definidos pelo professor, porém a equipe poderá sugerir algum tema lembrando que o tema, obrigatoriamente, deve ter uma aplicação direta dos conceitos de Disciplina e previamente aprovado pelo professor;

4) Todas as equipes devem apresentar a documentação por meio digital para euclerio.filho@ulife.com.br (constando: do roteiro da implementação e Artigo com os nomes dos componentes da equipe e número de matrícula) no dia 25 de novembro de 2026

5) Os trabalhados serão apresentados nos dias 25 de novembro de 2026 e 02 de dezembro para a turma matutina e 26 de novembro de 2026 e 03 de dezembro para a turma noturna; conforme divisão definidas pelo professor

6) O resultado final deste projeto deve ser apresentado em forma de slides, a explicação da tecnologia, o embasamento teórico, o modelo da arquitetura lógica e física do sistema, estudo de caso discussão final com possíveis resultados obtidos;

7) A apresentação final do projeto deverá ser realizada durante o horário de aula, presencialmente, cada equipe terá 20 minutos para apresentação.

8) Não serão permitidos projetos que use qualquer tipo de linguagem inadequada.

9) As apresentações deverão ser presenciais e realizadas 25 de novembro de 2026 e 02 de dezembro para a turma matutina e 26 de novembro de 2026 e 03 de dezembro para a turma noturna

PONTUAÇÃO:

A Pontuação consistirá em:
a) Apresentação do Projeto (de 0 até 10 pontos) – A apresentação final consistirá na apresentação em forma de *Slides* durante o horário da aula, seguindo o formato e com duração máxima de 20 min.

b) Relatório final (de 0 até 10 pontos) – O relatório final deverá ser enviado por email para euclerio.filho@ulife.com.br, seguindo a estrutura:

i Introdução.
ii Projeto (Apresentação do protótipo incluindo a representação gráfica, esquemas elétricos ou eletrônicos e/ou *prints* ou vídeos da simulação).
iii Resultados.
iv Conclusão.
v) Anexo: slides ppt e roteiro de instalação e configuração , prompt e nome usados na LLM, arquivos de configurações,

c) Implementação (de 0 até 10 pontos) – Deve ser apresentada em slides sobre o tem escolhido e demonstrado na sala.

TEMAS PARA O TRABALHO DE SISTEMAS OPERACIONAIS

Tema 1

Título: Orquestração de serviços daemon com Kubernetes e API de gerência de processos

Descrição: Construir um ambiente com três containers, cada um executando um serviço daemon distinto, sendo que um deles hospeda uma API em Python capaz de listar e gerenciar os processos em execução. Todo o conjunto é orquestrado pelo Kubernetes, demonstrando deploy, escalonamento e auto-recuperação.

Regras e detalhes:

Mínimo de 3 containers com daemons diferentes (ex.: nginx, um cron/agendador, e o container da API).
A API em Python deve expor pelo menos: listar processos, inspecionar um processo por PID, iniciar e encerrar um processo.
Demonstrar no Kubernetes: self-healing (matar um pod e mostrar recriação) e scaling (aumentar réplicas).
Entregar os arquivos YAML (Deployments/Services) versionados.

Tecnologias sugeridas: Docker/containerd; Minikube ou kind; kubectl; Python + FastAPI + psutil; nginx.

Tema 2

Título: Análise de sistema com Ollama, serviço gRPC e mensageria

Descrição: Usar um LLM local (Ollama) para analisar dados de sistema, expor essa análise por um serviço gRPC e distribuir os resultados por um protocolo de mensageria, formando um pipeline: coleta → análise por IA → transporte assíncrono.

Regras e detalhes:

O Ollama deve rodar 100% local, com um modelo aberto, recebendo dados reais do sistema (ex.: uso de CPU/memória, logs) e retornando uma análise em texto.
A comunicação entre coletor e serviço de análise deve usar gRPC com contrato .proto (ao menos um método unário e um de streaming).
Os resultados devem ser publicados numa fila/tópico de mensageria e consumidos por outro processo.
Demonstrar o fluxo ponta a ponta funcionando.

Tecnologias sugeridas: Ollama (llama3/mistral); Python ou Go; gRPC + Protocol Buffers; RabbitMQ, Apache Kafka ou MQTT (mosquitto); psutil para coleta.

Tema 3

Título: Auditoria e detecção de comprometimento em servidores Linux

Descrição: Auditar a segurança e a conformidade de servidores Linux, sendo que um dos servidores contém indícios de comprometimento simulados (amostra de teste benigna e artefatos de persistência forjados), que devem ser detectados e reportados pelas ferramentas de auditoria.

Regras e detalhes:

Mínimo de 2 servidores: um "limpo" e um com vírus.
Executar auditoria de conformidade e varredura por ameaças/rootkits, gerando relatório antes/depois de mitigar.
Entregar o relatório com as detecções e as recomendações de hardening.
Todo o lab deve rodar em VMs isoladas.

Tecnologias sugeridas: OpenSCAP (oscap + SCAP Security Guide); ClamAV; rkhunter/chkrootkit; auditd; VirtualBox/VMware.

Tema 4

Título: Cluster de computação distribuída em Linux com telemetria Grafana

Descrição: Implementar um cluster Linux voltado a computação paralela (execução de uma carga distribuída real entre os nós), supervisionado por telemetria completa em Grafana, mostrando o comportamento de cada nó durante o processamento.

Regras e detalhes:

Mínimo de 1 nó master + 2 workers, com SSH por chave e diretório compartilhado.
Executar uma tarefa paralela real (ex.: cálculo distribuído com MPI) e medir speedup variando o número de processos/nós.
Telemetria por nó no Grafana: CPU, memória, rede, com dashboards.
Entregar gráficos comparativos de desempenho.

Tecnologias sugeridas: OpenMPI/MPICH; NFS; Prometheus + node_exporter; Grafana; VMs Linux.

Tema 5

Título: Boot do Linux: GRUB 2 versus Unified Kernel Images (nmbl "No More Bootloader")

Descrição: Comparar o modelo tradicional de boot com GRUB 2 contra a proposta do Fedora de eliminar o bootloader usando UKIs — kernel, initramfs e cmdline assinados num único binário EFI —, avaliando segurança, simplicidade e manutenção.

Regras e detalhes:

Demonstrar na prática os dois fluxos de boot numa VM UEFI.
Gerar uma UKI e fazer o firmware bootá-la diretamente, sem GRUB.
Abordar a cadeia de confiança e o Secure Boot em cada modelo.
Entregar tabela comparativa (nº de componentes, complexidade de atualização, superfície de ataque, dual-boot).
Atenção: risco de quebrar o boot — trabalhar sempre em VM com snapshot.

Tecnologias sugeridas: VM UEFI; GRUB 2; systemd-boot; ukify; Fedora; sbctl/chaves de Secure Boot.

Tema 6

Título: Virtualização: cluster web com monitoramento de desempenho

Descrição: Montar um cluster em VMs para hospedar um serviço web simples (servidor HTTP + cliente que faz requisições) e implementar monitoramento coletando métricas de desempenho das VMs. (Enunciado original do professor.)

Regras e detalhes:

O cluster deve conter no mínimo: 1 ferramenta de monitoramento, 1 servidor HTTP e 1 cliente de requisições.
Gerar carga com o cliente e visualizar o impacto nas métricas.
Dashboards de CPU, memória, rede e latência/requisições no Grafana.
Entregar prints/dashboards mostrando o comportamento sob carga.

Tecnologias sugeridas: Hypervisor (VirtualBox, Hyper-V ou VMware); nginx ou Apache; Grafana (+ Prometheus/node_exporter); ferramenta de carga (ab, wrk ou k6).

Tema 7

Título: Comunicação entre aplicações Linux com D-Bus

Descrição: Explicar o D-Bus, o barramento de IPC usado por aplicações Linux para se comunicarem, demonstrando introspecção de serviços existentes e a criação de um serviço próprio com um cliente.

Regras e detalhes:

Explicar system bus vs session bus, objetos, interfaces, métodos e sinais.
Explorar serviços reais (ex.: consultar bateria via UPower, ou NetworkManager) por introspecção.
Criar um serviço próprio que exponha um método e um cliente que o chame e reaja a um sinal.
Entregar o código e a demonstração da troca de mensagens.

Tecnologias sugeridas: D-Bus; busctl/dbus-send/gdbus; Python (dbus-python ou pydbus); d-feet (opcional).

Tema 8

Título: Estrutura interna de sistemas de arquivos: o caso Ext2

Descrição: Estudar a organização on-disk de um sistema de arquivos usando o Ext2 (superblock, inodes, bitmaps, blocos, diretórios), inspecionando essas estruturas e, opcionalmente, implementando um leitor.

Regras e detalhes:

Criar uma imagem Ext2 e inspecionar suas estruturas com ferramentas.
Explicar como um inode aponta para os blocos de dados (diretos, indiretos, duplo/triplo indireto).
Nível mínimo: inspecionar com debugfs/dumpe2fs. Nível avançado (bônus): programa que lê a imagem crua e lista arquivos do diretório raiz.
Entregar o passo a passo com a análise das estruturas.

Tecnologias sugeridas: Linux; dd + loop device; mke2fs; dumpe2fs, debugfs; opcional C ou Python; FUSE (bônus).

Tema 9

Título: Escalonamento de processos com EEVDF

Descrição: Explicar e demonstrar o escalonador EEVDF (Earliest Eligible Virtual Deadline First), que substituiu o CFS como padrão do kernel Linux, analisando seu comportamento com diferentes prioridades e cargas.

Regras e detalhes:

Explicar os conceitos de lag, elegibilidade e virtual deadline, comparando com o antigo CFS.
Realizar um experimento com cargas concorrentes, medindo latência de escalonamento e justiça entre processos.
Variar nice/latência e mostrar o efeito nos resultados.
Entregar as medições com perf/trace.
Tecnologias sugeridas: kernel ≥ 6.6; stress-ng; chrt/nice; perf sched, trace-cmd.

Tema 10

Título: Linux em smartphones: distribuições móveis não-Android

Descrição: Apresentar o estado do Linux "convencional" em celulares (não-Android), sua arquitetura de software e os desafios de rodar um SO desktop-like em hardware móvel, com análise comparativa entre as principais distros.

Regras e detalhes:

Comparar postmarketOS, Mobian e Ubuntu Touch (não apenas descrever uma) — maturidade, hardware suportado, interface, telefonia.
Explicar a stack: kernel + Wayland + shell móvel + camada de telefonia.
Demonstração em emulador ou hardware suportado (se disponível).
Entregar quadro comparativo com prós/contras.

Tecnologias sugeridas: postmarketOS/Mobian/Ubuntu Touch; Phosh ou Plasma Mobile; ModemManager/oFono; QEMU ou PinePhone.

Tema 11

Título: Sistemas Operacional de tempo real com FreeRTOS

Descrição: Apresentar o FreeRTOS, um SO de tempo real para microcontroladores, demonstrando conceitos de RTOS — tarefas, escalonamento preemptivo por prioridade, filas e semáforos — em uma aplicação com múltiplas tasks.

Regras e detalhes:

Explicar RTOS vs SO de propósito geral (determinismo, latência garantida, preempção).
Implementar 2–3 tasks com prioridades diferentes e comunicação entre elas via fila.
Rodar em hardware (ESP32/STM32) ou, na ausência, no port POSIX/simulador.
Entregar o código e a demonstração do escalonamento.

Tecnologias sugeridas: FreeRTOS; ESP32/STM32 ou FreeRTOS POSIX port/QEMU; ESP-IDF, PlatformIO ou Arduino.

Tema 12

Título: FreeBSD: um Unix além do Linux (ZFS - Z File System e jails)

Descrição: Apresentar o FreeBSD como um Unix completo distinto do Linux — diferenças de arquitetura, licença e sistema base unificado — com demonstração de recursos característicos como ZFS e jails.

Regras e detalhes:

Comparar filosofias Linux vs FreeBSD (kernel + userspace, licenças GPL vs BSD, base unificada).
Demonstrar ZFS (pool, dataset, snapshot, rollback) e jails (ambiente isolado — precursor dos contêineres).
Mostrar o gerenciamento de pacotes (pkg/ports).
Entregar a demonstração numa VM.

Tecnologias sugeridas: FreeBSD (VM); ZFS; jails; pkg; opcional bhyve.